The pattern of interactions in the Razor Page of listing 13.5 shows a common pattern. The page handler is the central controller for the Razor Page. It receives an input from the user (the `category` method parameter); calls out to the "brains" of the application (the `ToDoService`); and passes data (by exposing the `Items` property) to the Razor view, which generates the HTML response. If you squint, this pattern looks like the MVC design pattern.

Depending on your background in software development, you may have come across the MVC pattern in some form. In web development, MVC is a common paradigm, used in frameworks such as Django, Rails, and Spring MVC. But as it's such a broad concept, you can find MVC in everything from mobile apps to rich-client desktop applications. I hope that indicates the benefits of the pattern when it's used correctly! In section 13.3 we'll look at the MVC pattern in general and how ASP.NET Core uses it.

13.3 Understanding the MVC design pattern

The MVC design pattern is a common pattern for designing apps that have UIs. The MVC pattern has many interpretations, each of which focuses on a slightly different aspect of the pattern. The original MVC design pattern was specified with rich-client graphical user interface (GUI) apps in mind, rather than web applications, so it uses terminology and paradigms associated with a GUI environment. Fundamentally, though, the pattern aims to separate the management and manipulation of data from its visual representation.

Before I dive too far into the design pattern itself, let's consider a typical Razor Pages request. Imagine that a user requests the Razor Page from listing 13.5 that displays a to-do list category. Figure 13.7 shows how a